# Padrões de Arquitetura - iOS

Instrutor: Ralf Schweder de Lima



# A Importância dos Padrões de Arquitetura

Construir um aplicativo sem uma arquitetura definida é como erguer uma casa sem uma planta estruturada: você até consegue levantar as primeiras paredes, mas logo o teto começará a ceder.

No início de um projeto, juntar toda a lógica em um único arquivo parece o caminho mais rápido e fácil. Porém, conforme novas telas e funcionalidades são adicionadas, esse código desorganizado se transforma em um grande "emaranhado de problemas".

# A Importância dos Padrões de Arquitetura

A arquitetura de software é o conjunto de regras e padrões que determina onde cada pedaço de código deve morar e como as diferentes partes do aplicativo se comunicam.

Em sala de aula e no mercado de trabalho, entender e aplicar uma arquitetura sólida é fundamental por quatro motivos principais

# 1. Escalabilidade

Permite que o projeto cresça de tamanho e ganhe novas telas sem que as funcionalidades antigas quebrem misteriosamente.

Novos desenvolvedores conseguem entrar na equipe e entender onde mexer imediatamente.

# 2. Testabilidade

Quando a lógica de negócios está misturada com o layout da tela, testar o aplicativo se torna uma tarefa quase impossível.

Ao separar as responsabilidades, podemos testar os cálculos e as validações (como a conversão de valores e cálculos) de forma isolada e automatizada.

# 3. Reutilização de Código

Componentes visuais bem isolados e regras de negócio bem definidas podem ser reaproveitados em diferentes partes do aplicativo, ou até mesmo em outros projetos, reduzindo o retrabalho e o tempo de desenvolvimento.

# 4. Manutenção

O desenvolvedor sabe exatamente em qual arquivo deve ir para alterar essa regra, sem correr o risco de estragar o design visual da listagem ou do formulário.

# Conhecendo as Arquiteturas de Software para iOS



# 1. MVC (Model-View-Controller - O Padrão Tradicional)

O MVC foi o padrão arquitetural oficial recomendado pela Apple durante anos na era do UIKit. Ele tenta separar o aplicativo em três grandes responsabilidades:

- Model: Representa os dados puros e as regras de negócio (ex: o objeto Aluno).
- View: Os elementos visuais com os quais o usuário interage (telas, botões, labels criados via Storyboard ou código).
- Controller: O intermediário. Ele escuta as interações da View, busca ou altera dados na Model e atualiza a View manualmente.

# 1. MVC (Model-View-Controller - O Padrão Tradicional)

O grande desafio do MVC: No iOS, a View e o Controller acabam ficando fortemente acoplados na classe UIViewController.

Como a ViewController precisa gerenciar o ciclo de vida da tela, fazer requisições de API, tratar cliques e formatar textos, ela se transforma em um arquivo gigantesco.

Esse fenômeno ficou conhecido na comunidade como Massive View Controller (View Controller Massiva), gerando códigos difíceis de testar e manter.

# 2. VIPER (Muito comum em grandes empresas / Legados)

É uma arquitetura extremamente segmentada e baseada nos princípios de responsabilidade única. É famosa por dividir uma única tela em 5 partes:

- View: Exibe a interface e recebe os toques do usuário.
- Interactor: Contém a lógica de negócio pura (fazer requisições, buscar dados).
- Presenter: Pega os dados do Interactor, formata-os e manda para a View. É o "maestro".
- Entity: As Models puras do projeto.
- Router: Cuida estritamente da navegação (qual é a próxima tela).

# 3. VIP (Clean Swift)

Uma evolução natural que nasceu na comunidade iOS para simplificar o VIPER e corrigir problemas de acoplamento do Presenter.

Ela funciona em um fluxo unidirecional em formato de ciclo.

A View avisa a ação para o Interactor ➡️ O Interactor processa a lógica de negócios e passa o resultado para o Presenter ➡️ O Presenter formata o dado para exibição e atualiza a View.

# 4. MVVM (Model-View-ViewModel - O Padrão Moderno)

O MVVM nasceu para resolver o principal problema do MVC, retirando toda a "lógica de apresentação" de dentro da tela e movendo para uma nova camada chamada ViewModel:

- Model: Mantém o mesmo papel (dados puros como a struct Aluno).
- View: No SwiftUI, a View torna-se puramente declarativa. Ela é apenas uma casca visual que reflete o estado atual dos dados.
- ViewModel: O "cérebro" da tela. Ela prepara os dados para a View exibir (ex: converte as notas em String para Double, calcula a média e valida campos).

# 4. MVVM (Model-View-ViewModel - O Padrão Moderno)

O grande diferencial do MVVM: A comunicação entre a View e a ViewModel ocorre por meio de data binding (vínculo de dados automático).

A View observa a ViewModel. Sempre que uma variável da ViewModel muda (graças ao mecanismo do @Observable ou ObservableObject), a View se redesenha sozinha. Isso elimina a necessidade de um controlador ditar manualmente o que a tela deve exibir, tornando o código altamente modular, limpo e extremamente amigável para testes unitários.

# 4. MVVM (Model-View-ViewModel - O Padrão Moderno)

Dependendo da complexidade do projeto, novas camadas podem surgir. Veja abaixo as mais usuais:

- 📁 FormModels: Estruturas específicas para capturar, validar e manipular dados de formulários antes de virarem Models.
- 📁 Services (ou Networking): Responsável por buscar dados externos (chamadas de API HTTP, Firebase, etc.).
- 📁 Repositories (Opcional, para projetos médios/grandes): Uma camada que decide de onde os dados vêm.

# 4. MVVM (Model-View-ViewModel - O Padrão Moderno)

Dependendo da complexidade do projeto, novas camadas podem surgir. Veja abaixo as mais usuais:

- 📁 Database (Storage): Persistência de dados local.
- 📁 Components (Widgets): Elementos visuais reutilizáveis em várias telas do app.
- 📁 Extensions: Extensões de classes nativas para facilitar o código.
- 📁 Resources (Assets): Arquivos de configuração, cores, imagens (Assets.xcassets), fontes customizadas e arquivos de tradução (Localizable.strings).

# 5. TCA (The Composable Architecture - O mais moderno)

Se o MVVM é o padrão atual da Apple para SwiftUI, o TCA é a arquitetura de comunidade que mais cresce no mundo hoje para projetos SwiftUI modernos.

Ela é fortemente inspirada pelo Redux (famoso no ambiente Web/React) e baseia-se em Estado Global Unificado.

O app possui um State (estado com todos os dados), uma Action (lista de ações que o usuário pode fazer) e um Reducer (uma função pura que pega a ação e diz exatamente como o estado deve mudar).

# Vamos praticar!

